

Ofren Collect — Test Cases

Project: Ofren Collect — Recurring billing + automatic reconciliation on Monnify **Author:** Ofren Technologies **Context:** APIConf Lagos 2026 × Monnify Developer Challenge **Companion to:** REQUIREMENTS.md, USER_STORIES.md

How to read this

Each test case has an ID, the requirement/story it verifies, preconditions, steps, expected result, and priority. **P1** = core demo path (must pass before submission); **P2** = important; **P3** = edge/robustness (nice to verify, don't block on).

Status column is for you to fill during testing: Pass / Fail / Not run / N/A.

Legend for setup: "sandbox payment" = a simulated inflow into a reserved account via the Monnify sandbox.

0. Authentication & multi-tenancy

Tenant isolation is a security property, so these are P1 — a leak here is worse than a broken feature. "Tenant A / Tenant B" means two separately-registered businesses.

ID	Verifies	Priority	Preconditions	Steps	Expected result	Status
TC-0.1	FR-0.1	P1	App running	Register a new business with owner email + password	A tenant and an owner user are created; can log in immediately	
TC-0.2	FR-0.2, FR-0.3	P1	A registered user	Log in with correct credentials	A signed token is returned carrying the user's tenant and role	
TC-0.3	FR-0.3	P1	Registered user	Inspect stored password (DB)	Stored only as a salted hash; never plain text	
TC-0.4	FR-0.2	P2	Registered user	Log in with wrong password	Rejected; message doesn't reveal which field was wrong	
TC-0.5	FR-0.4	P1	No token	Call any business endpoint (e.g. GET /api/plans) without a token	Rejected with 401	
TC-0.6	FR-0.4	P1	No token	Open a SignalR hub connection without a token	Connection rejected	
TC-0.7	FR-0.5, FR-0.6	P1	Tenant A logged in; Tenant B has data	Tenant A lists plans/customers/subscriptions	Only Tenant A's records returned; none of Tenant B's	
TC-0.8	FR-0.5	P1	Tenant A logged in	Attempt to fetch a Tenant B record by its known id (e.g. GET /api/subscriptions/{B-id})	Not found / forbidden — never returns another tenant's record	
TC-0.9	FR-0.6	P1	Tenant A token	Craft a request trying to set/spoof a different tenant id in body or header	Ignored — tenant is taken from the token only, not client input	
TC-0.10	FR-0.7	P1	Tenant A and Tenant B both connected via SignalR	A payment reconciles for Tenant A	Only Tenant A's client receives the event; Tenant B receives nothing	
TC-0.11	FR-0.8	P1	A subscription exists under Tenant A	Monnify webhook (no token) for that reserved account	Resolves to Tenant A via the reserved account; records stamped with Tenant A	
TC-0.12	FR-0.9	P2	A non-owner (Staff) token	Call GET /api/audit	Rejected — Owner role required	
TC-0.13	FR-0.6	P2	Tenant A logged in	Global query filter check: run a read that forgets to filter (dev-level)	Filter still scopes to Tenant A automatically	

ID	Verifies	Priority	Preconditions	Steps	Expected result	Status
TC-1.1	FR-1.1, US-1.1	P1	App running	Create a plan: name "Basic", amount 5000, interval Monthly	Plan is saved and appears in the plan list with correct values	
TC-1.2	FR-1.2, US-1.2	P1	≥1 plan exists	Open plan list	All plans shown with name, amount, interval	
TC-1.3	FR-1.3	P2	App running	Create a plan with amount 0	Rejected with a clear validation message; no plan saved	
TC-1.4	FR-1.3	P2	App running	Create a plan with amount -100	Rejected; no plan saved	
TC-1.5	FR-1.3	P2	App running	Create a plan with blank name	Rejected; no plan saved	
TC-1.6	FR-1.3	P3	App running	Create a plan with a very large amount (e.g. 999,999,999)	Either accepted and stored precisely, or rejected with a clear limit — no overflow or silent truncation	
TC-1.7	FR-1.1	P3	App running	Create a plan with a decimal amount (e.g. 4999.99)	Amount stored to 2 decimal places without rounding error	
TC-1.8	FR-1.1	P3	App running	Create a plan with name containing special characters/emoji	Stored and displayed without breaking layout or throwing	
TC-1.9	FR-1.4	P3	A plan exists	Deactivate the plan; attempt to enrol a customer on it	Plan cannot be used for new subscriptions	

2. Customer & subscription management

ID	Verifies	Priority	Preconditions	Steps	Expected result	Status
TC-2.1	FR-2.1, US-2.1	P1	App running	Register customer "Chidi", email valid	Customer saved and selectable for enrolment	
TC-2.2	FR-2.1	P2	App running	Register customer with malformed email (e.g. "chidi@")	Rejected with validation message	
TC-2.3	FR-2.1	P3	App running	Register customer with blank name	Rejected	
TC-2.4	FR-2.2, FR-2.3, US-2.2	P1	A plan and a customer exist	Enrol the customer on the plan	Subscription created; a Monnify reserved account is provisioned; account number + bank persisted and displayed	
TC-2.5	FR-2.4	P2	Enrolment succeeded	Inspect the subscription's account reference	A unique internal reference exists and was sent to Monnify	
TC-2.6	FR-2.5, US-2.3	P1	Enrolment succeeded	Check invoices for the new subscription	One invoice exists with the plan amount and a due date derived from the interval	
TC-2.7	FR-2.3	P2	Two customers enrolled on the same plan	Compare their reserved accounts	Each subscription has a distinct reserved account number	
TC-2.8	FR-2.3, NFR-2.2	P2	Monnify sandbox unreachable / returns error on account creation	Attempt enrolment	Enrolment fails gracefully with a clear message; no orphaned subscription left in a broken state	
TC-2.9	FR-2.2	P3	A customer already enrolled on a plan	Enrol the same customer on the same plan again	Handled deliberately — either prevented with a message or creates a clearly separate subscription; no silent duplicate account confusion	
TC-2.10	FR-2.7, US-2.4	P2	An active subscription exists	Cancel it	No new invoices are generated; subscription shown as cancelled	
TC-2.11	FR-2.7	P3	A cancelled subscription	A sandbox payment arrives into its (old) reserved account	Handled without error; documented behaviour (recorded as unmatched/credit, not crash)	

3. Payment detection & verification (webhooks)

ID	Verifies	Priority	Preconditions	Steps	Expected result	Status
TC-3.1	FR-3.1, FR-3.2, US-3.2	P1	Webhook endpoint exposed to Monnify	Trigger a sandbox payment into a subscription's reserved account	Webhook is received and responds HTTP 200 promptly	
TC-3.2	FR-3.3, US-3.3	P1	Webhook endpoint live	Send a webhook with an invalid/missing signature	Request is rejected (fails closed); no reconciliation occurs	
TC-3.3	FR-3.4, US-3.3	P1	Valid webhook received	Observe processing	Transaction is independently verified with Monnify before any invoice is updated	
TC-3.4	FR-3.5	P1	A verified payment	Inspect stored records	Transaction reference, amount, paid-into account, and timestamp are recorded	
TC-3.5	FR-3.6, NFR-2.1, US-3.4	P1	A payment already processed	Re-deliver the exact same webhook (same reference)	No double counting; invoice amount paid is unchanged on the second delivery	
TC-3.6	FR-3.4, NFR-2.2	P2	Webhook received but Monnify verification endpoint errors/times out	Observe	Processing does not crash; notification is not silently lost; retry-safe or surfaced	
TC-3.7	FR-3.1	P2	Webhook endpoint live	Send a malformed/empty body	Returns a controlled response; does not throw an unhandled 500 that would trigger endless retries	
TC-3.8	FR-3.3	P3	Webhook endpoint live	Send a webhook with a tampered amount but otherwise plausible payload, invalid signature	Rejected at signature stage before amount is trusted	
TC-3.9	FR-3.6	P3	Endpoint live	Send two identical webhooks concurrently (race)	Only one is applied; no double count from the race	
TC-3.10	FR-3.2	P3	Endpoint live	Verification/processing is slow	200 is still returned promptly (ack decoupled from heavy processing)	
TC-3.11	NFR-2.6	P2	A webhook acknowledged, then the app is killed mid-processing	Restart the app	The inbox row is picked up and processing completes; the acknowledged payment is not lost	
TC-3.12	NFR-2.6	P3	Webhook processing in flight	Force a failure after the inbox write but before reconciliation	On retry, the payment reconciles exactly once (inbox + idempotency together)	

4. Reconciliation

ID	Verifies	Priority	Preconditions	Steps	Expected result	Status
TC-4.1	FR-4.1, FR-4.2, US-4.1	P1	Subscription with open invoice (¥5,000 due)	Sandbox payment of ¥5,000 into its reserved account	Payment resolved to the correct subscription and applied to its open invoice	
TC-4.2	FR-4.3, US-4.2	P1	Open invoice ¥5,000 due	Pay exactly ¥5,000	Invoice marked Paid	
TC-4.3	FR-4.4, US-4.3	P1	Open invoice ¥5,000 due	Pay ¥3,000	Invoice marked Underpaid ; outstanding balance ¥2,000 recorded	
TC-4.4	FR-4.4	P2	Invoice Underpaid, ¥2,000 balance	Pay a further ¥2,000	Cumulative paid = ¥5,000; invoice becomes Paid	
TC-4.5	FR-4.5, US-4.4	P2	Open invoice ¥5,000 due	Pay ¥6,000	Invoice marked Overpaid ; ¥1,000 credit recorded	
TC-4.6	FR-4.6, US-4.5	P2	Invoice fully paid	Observe subscription	Subscription advances to next period; a new invoice is created with correct next due date	
TC-4.7	FR-4.7, US-4.6	P2	A reserved account not linked to any active subscription	Sandbox payment into it	Recorded as Unmatched and surfaced for review; not discarded, no crash	
TC-4.8	FR-4.8, US-4.7	P2	Invoice past its due date, unpaid	Run the overdue sweep (or advance clock)	Subscription flagged Overdue	
TC-4.9	FR-4.2	P3	Open invoice ¥5,000	Pay ¥0.01 (tiny amount)	Recorded as Underpaid with correct balance; no rounding to zero	
TC-4.10	FR-4.2	P3	Two open invoices across two subscriptions, same amount	Pay into each account	Each payment lands on the correct invoice — amount alone never causes cross-matching (account is the key)	
TC-4.11	FR-4.6	P3	Invoice paid, next invoice created	Overpayment credit from previous cycle exists	Credit is applied/visible against the next invoice, not lost	
TC-4.12	FR-4.3	P3	Fully paid subscription	A late duplicate real payment arrives for the same period	Handled deliberately (credit or unmatched), not applied as a second "paid" on a settled invoice	
TC-4.13	FR-4.1	P3	Payment references an account number not in our system at all	Inflow processed	Unmatched, surfaced, no exception	
TC-4.14	FR-0.8, FR-4.1	P1	Subscriptions under Tenant A and Tenant B, same plan amount	Pay into each tenant's reserved account	Each payment reconciles under the correct tenant; PaymentEvent stamped with the right TenantId; neither tenant sees the other's	
TC-4.15	FR-0.7	P2	Tenant A and B dashboards open	A reconciliation for Tenant A	Only Tenant A's dashboard updates live; Tenant B's is unaffected	

5. Dashboard & reporting

ID	Verifies	Priority	Preconditions	Steps	Expected result	Status
TC-5.1	FR-5.1, US-5.1	P1	≥1 subscription	Open dashboard	Each subscription shows customer, plan, reserved account, next due date, status	
TC-5.2	FR-5.4	P1	App running	Create a plan and enrol a customer from the dashboard	Both actions work from the dashboard UI	
TC-5.3	FR-5.2, US-5.2	P2	Subscriptions in different states	View dashboard	Paid / Underpaid / Overdue are visually distinct	
TC-5.4	FR-5.3, US-5.3	P2	Some paid, some overdue, ≥1 unmatched	View summary strip	Total collected this period, overdue count, and unmatched count are correct	
TC-5.5	FR-5.1	P3	No subscriptions yet (fresh, unseeded)	Open dashboard	Empty state renders cleanly, no errors, with guidance to create a plan	
TC-5.6	NFR-5.2	P3	~100 subscriptions seeded	Open dashboard	List renders responsively without noticeable lag	

6. Settlement (stretch)

ID	Verifies	Priority	Preconditions	Steps	Expected result	Status
TC-6.1	FR-6.1, US-7.1	P3	Collected funds exist	Trigger a settlement transfer to a merchant account (sandbox)	Transfer initiated via Monnify; recorded with reference and status	
TC-6.2	FR-6.2	P3	Settlement attempted while Monnify transfer errors	Observe	Failure recorded/surfaced; no false "settled" state	

6A. AI assistant (stretch, flag-gated)

ID	Verifies	Priority	Preconditions	Steps	Expected result	Status
TC-6A.1	FR-7.1, US-7.2	P3	Tenant with known totals; AI enabled	Ask "how much did I collect this week?"	Answer matches the known figure for that tenant	
TC-6A.2	FR-7.3	P1	AI enabled	Ask the assistant to do something destructive ("cancel Chidi's subscription", "refund X")	Refuses / cannot act — read-only; no state change occurs	
TC-6A.3	FR-7.3	P2	AI enabled	Ask a question whose answer isn't in the data	Says it can't answer; does not invent a figure	
TC-6A.4	FR-7.5, FR-0.6	P1	Tenant A and Tenant B both have data; AI enabled	As Tenant A, ask about collections	Answer reflects only Tenant A's data; never references Tenant B	
TC-6A.5	FR-7.4	P1	AI feature flag OFF	Call POST /api/assistant/ask	Endpoint disabled/unavailable; the rest of the app works unchanged	
TC-6A.6	FR-7.4	P2	Provider swapped in config (e.g. local model)	Ask a question	Works without code change — provider-agnostic	
TC-6A.7	FR-7.5	P2	AI enabled	Ask several questions rapidly	Assistant calls are rate-limited and audit-logged like any endpoint	
TC-6A.8	FR-7.2	P2	AI unreliable in testing	Decision point	Feature is switched off for submission rather than shipped half-working	

7. Security, audit, rate limiting & resilience

ID	Verifies	Priority	Preconditions	Steps	Expected result	Status
TC-7.1	NFR-1.1, US-8.3	P1	Repo prepared	Grep working tree and git history for keys/secrets	No API key, secret key, contract code, or JWT signing key present anywhere	
TC-7.2	NFR-1.2	P1	Repo prepared	Inspect repo for config	Only <code>.env.example</code> with placeholders is committed; real <code>.env</code> is gitignored	
TC-7.3	NFR-1.3	P1	App configured	Confirm base URL and keys	Only Monnify sandbox endpoints/keys are used; no live credentials	
TC-7.4	NFR-1.6	P2	App running with logging	Trigger auth + a payment	Logs contain no full secret values or full sensitive payloads	
TC-7.5	NFR-2.4	P2	App running	Make several Monnify calls in quick succession	Auth token is cached and reused; not re-fetched on every call	
TC-7.6	NFR-1.4	P1	Endpoint live	(Covered by TC-3.2) unauthenticated webhook	Rejected	
TC-7.7	NFR-1.8	P1	App configured	Confirm JWT signing key source	Signing key comes from config/secrets only; not in repo or frontend	
TC-7.8	NFR-1.7	P1	Tenant A logged in	Verify isolation is enforced at the data layer (global query filter), not per-query	A read written without an explicit tenant filter still returns only Tenant A's rows	
Audit						
TC-7.9	FR-8.1	P1	Logged-in tenant	Make several API calls (create plan, enrol, view dashboard)	Each call produces an audit entry with correlation id, user, tenant, method, path, status, timestamp	
TC-7.10	FR-8.2	P1	Audit logging on	Call login and an endpoint carrying account data	Audit entries redact password, tokens, secrets, and full account numbers	
TC-7.11	FR-8.3	P2	Tenant A (Owner) with activity; Tenant B has activity	Tenant A queries GET /api/audit	Only Tenant A's audit entries returned; never Tenant B's	
TC-7.12	FR-8.4	P2	A request made	Compare response header, audit entry, and logs	Same correlation id links all three	
Rate limiting						
TC-7.13	NFR-1.9	P2	Authenticated tenant	Exceed the per-tenant request limit on an API endpoint	Excess requests get 429 with a retry hint; limit is per-tenant	
TC-7.14	NFR-1.9	P2	Anonymous	Hammer POST /api/auth/login from one IP	Rate-limited per IP with 429 (brute-force protection)	
TC-7.15	NFR-1.9	P3	Two tenants active	Tenant A hits its limit	Tenant B is unaffected — limits are isolated	
Resilience						
TC-7.16	NFR-2.5	P2	Monnify returns transient errors	Make a call that fails transiently	Retried with backoff; succeeds if Monnify recovers within retries	
TC-7.17	NFR-2.5	P2	Monnify down for a sustained period	Repeated calls	Circuit breaker opens (fails fast); doesn't hang or pile requests; half-opens to probe recovery	
TC-7.18	NFR-2.2	P3	Circuit breaker open	An enrolment is attempted	Fails cleanly with a clear message; no corrupt/orphaned state	

8. Setup, portability & submission readiness

ID	Verifies	Priority	Preconditions	Steps	Expected result	Status
TC-8.1	NFR-3.1, US-8.1	P1	Fresh machine / clean clone	Follow README steps only	App builds and runs with no undocumented steps	
TC-8.2	NFR-4.1	P1	Clean clone	Run app first time	PostgreSQL (local container or the configured managed Postgres) is reachable; EF Core migrations create the schema and seed automatically	
TC-8.3	NFR-3.2, US-8.2	P1	First run	Log in with the seeded account and open the dashboard	A seeded tenant, plan, customer, and subscription are present (app not empty)	
TC-8.4	NFR-6.2, US-8.3	P1	Docs prepared	Click every link in README and docs	No broken links	
TC-8.5	NFR-4.3	P2	.NET 10 SDK on a second OS	Build and run	Runs on at least one OS other than the primary dev machine	
TC-8.6	NFR-3.3, US-8.4	P1	README complete	Read as a first-time evaluator	Problem, solution, and how-it-works are clear within ~2 minutes	
TC-8.7	NFR-6.1	P1	Repo public	Access repo URL logged out	Public repo with step-by-step setup guide is reachable	
TC-8.8	NFR-5.1	P2	App running	Send a webhook	Endpoint acknowledges within a few seconds	

9. Submission governance (non-code, but required to qualify)

ID	Verifies	Priority	Steps	Expected result	Status
TC-9.1	NFR-6.3	P1	Confirm submission time	Submitted before 12:00 WAT, 21 July 2026	
TC-9.2	NFR-6.4	P1	Check social post	Posted with #APIConfXMonnify and #DeveloperChallenge	
TC-9.3	NFR-6.5	P1	Confirm team	Exactly two members, both registered in-person attendees	
TC-9.4	—	P1	Slack	Submission made via the link in the apiconf-hackathon channel	

Test execution priority (3-day reality)

The scope now spans core reconciliation, security, and operational concerns, so run tests in tiers matching the build order. **Every P1 must pass before submission.**

Tier 1 — core reconciliation (the demo):

TC-1.1, TC-1.2, TC-2.1, TC-2.4, TC-2.6, TC-3.1-3.5, TC-4.1-4.3, TC-5.1, TC-5.2

Tier 2 — security & isolation (P1 — a tenant leak is worse than a broken feature):

TC-0.1-0.3, TC-0.5-0.11, TC-4.14, TC-7.7, TC-7.8

Tier 3 — operational (audit, resilience):

TC-7.9, TC-7.10 (audit) · TC-3.11 (durable webhooks) · TC-7.13, TC-7.14 (rate limiting) · TC-7.16, TC-7.17 (retry/breaker)

Always — submission & repo hygiene:

TC-7.1-7.3, TC-8.1-8.4, TC-8.6, TC-8.7, TC-9.1-9.4

P2 cases raise quality; run them once P1 is green. P3 edge cases prove robustness — verify the ones near the reconciliation engine (TC-4.9–4.15, TC-3.9, TC-3.12) if time allows, but never at the cost of a passing tier above.

Notes on edge-case philosophy

Two families of edge case carry the most weight now.

Money correctness: idempotency (TC-3.5, TC-3.9), same-amount cross-matching (TC-4.10), partial-then-complete payment (TC-4.4), unmatched inflows (TC-4.7, TC-4.13), and durable webhooks (TC-3.11, TC-3.12). These are the failure modes a real reconciliation system must survive.

Tenant isolation: cross-tenant reads (TC-0.7, TC-0.8), tenant spoofing (TC-0.9), scoped real-time (TC-0.10, TC-4.15), webhook tenant resolution (TC-0.11), and AI staying in-tenant (TC-6A.4). A leak here is a security failure, not a bug — treat these as non-negotiable P1s. Demonstrating both families is your strongest "technical depth" signal. Cosmetic edge cases (emoji names, huge numbers) matter far less — don't over-invest there.